Jose L. Muñoz
Juanjo Alins
Jorge Mata
Oscar Esparza

UPC Telematics Department

Internet Protocol

Contents

0.1 Practices 3

0.1 Practices

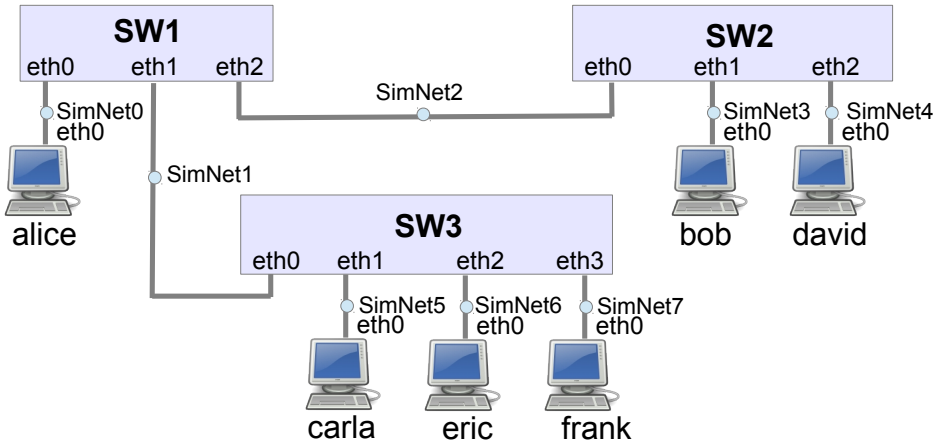


Figure 1: Scenario for testing switching and routing

Note. In the practices we always use classless IP (with network masks).

Exercise1– In this exercise we will examine how Ethernet switching and IP routing work together. We will use the simulation scenario of Figure 1. Execute on your **phyhost** the following command to start this scenario with its initial configuration:

```
telematic@phyhost:~$ simctl switching-vlan start
```

Remember that when you finish the exercise you have to stop the scenario.

1. Using the IP address block 192.168.100.0/24 assign an IP address to **bob** and **frank**. Example:

   ```
   root@frank:~# ifconfig eth0 192.168.100.5/24
   ```

   Capture on `SimNet1`, `SimNet2` and `SimNet3` and try a ping from **bob** to **frank** of one message (use the IP that you have assigned to **frank**):

   ```
   root@bob:~# ping -c 1 192.168.100.5
   ```

   Describe in detail your configuration and all the processes involved that make the `ping` command work. Include in your explanation the MACs learned by the switches, the ARP caches and the MAC (and IP addresses) in each frame captured.

2. Now we will convert SW1, which is a Linux system into a router. To implement this configuration you have to:

   - Delete the bridge (switch) in SW1:

     ```
     root@SW1:~# ifconfig br1 down
     root@SW1:~# brctl delbr br1
     ```

   - Make the Linux system SW1 to behave as router (enable IPv4 packet forwarding). This is accomplished typing:

     ```
     root@SW1:~# echo 1 > /proc/sys/net/ipv4/ip_forward
     ```

     Note. If you want to disable IPv4 packet forwarding, you can type:

     ```
     # echo 0 > /proc/sys/net/ipv4/ip_forward
     ```

   - Note that the router breaks the topology in three different L2 networks.
   - Since now SW1 is a router, it is mandatory that you assign a proper IP address to each interface of SW1.
   - For each L2 network, use the IP networks 192.168.X.0/24 with X=100,101,102.
   - You will have to define routing entries in the hosts. To do so, use the `route` command. An example of how to use the `route` command is the following:

     ```
     # route add -net 192.168.101.0/24 gw 192.168.100.1
     ```

     The previous command means that to forward traffic to any address of the IP network 192.168.101.0 (which uses 24/8 for netID/hostID) you must use a router with IP address 192.168.100.1. It is remarkable that for the previous command to work, the router must be directly reachable.

   - To check a routing table type:

     ```
     # route -n
     ```

   Capture on `SimNet1`, `SimNet2` and `SimNet3` and try a `ping` of one message from **alice** to **bob** and another `ping` of one message from **bob** to **frank**.

   Describe in detail the configuration that you propose and all the processes involved that make the `pings` work. Include in your explanation the MACs learned by the switches, the ARP caches and the MAC (and IP addresses) in each frame captured. Explain also the differences with the previous case, when SW1 was a switch.

Exercise2– In this first exercise, we will examine how the direct forwarding of IP datagrams works. We will use the virtual network topology shown in Figure 2, which has a hub and four virtual machines: **virt1**, **virt2**, **virt3** and **virt4**. Type on your physical host the following command to start the scenario:

```
phyhost$ simctl subnetting start
```

Once the scenario is started, you can get a terminal on each virtual machine using the "get" parameter of `simctl`:

```
phyhost$ simctl subnetting get virt1
```

1. Analyzing the IP addresses assigned to **virtX** machines in the network (see Figure 2), find which is the larger mask (biggest quantity of ones) that makes all the machines on the topology belong to the same IP network. In each virtual machine, use `ifconfig` to configure the IP address and the mask found.

2. Capture on the **phyhost** the `SimNet0` with `wireshark`. Check that the ARP cache is empty in **virt1**:

   ```
   virt1$ arp -n
   ```

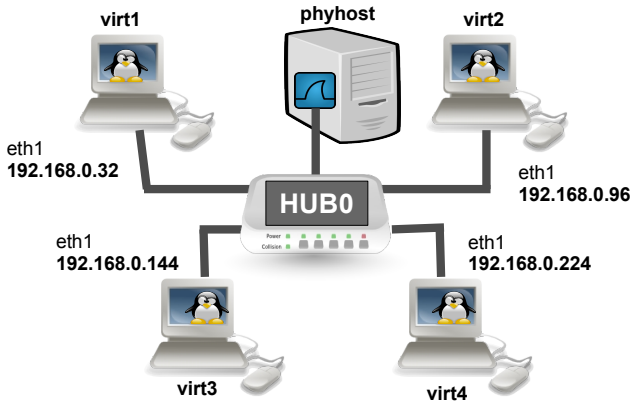


Figure 2: Scenario "subnetting".

From **virt1**, execute a `ping` with the options necessary to send only one *icmp-request* message to **virt2**. Wait a few seconds and run again the previous `ping` command. Explain the status of the ARP caches of all the virtual machines and the differences between the captured frames for the first and second `ping`.

Note. Observe that Linux generates a gratuitous ARP some time after the end of each transmission. These gratuitous ARPs are unicast and they are intended for refreshing the ARP cache.

3. Now, let's delete the ARP entry for 192.168.0.96 in **virt1**:

```
virt1# arp -d 192.168.0.96
```

Execute the previous `ping` command again and explain the traffic captured.

4. Now, let's create an erroneous mapping for 192.168.0.96 in **virt1**:

```
virt1# arp -s 192.168.0.96 00:70:48:29:5c:99 temp
```

To see the behavior of this misconfiguration execute a `ping` from **virt1** to 192.168.0.96 but in this case sending two *icmp-request* messages with an interval of 8 seconds between these messages (see the man page of `ping`). Discuss the traffic captured.

5. Now, we need to "clean" the ARP cache of **virt1**:

```
virt1# ip neigh flush all
```

Next, you have to find out which is the mask needed to divide the network into two subnets so that **virt1** and **virt2** belong to one subnet and **virt3** and **virt4** belong to another subnet. Configure the IP/mask on each virtual machine and explain how you check the configuration.

6. Which would be the smallest mask (minimum number of ones) that makes not possible the IP communication between the machines on the topology?

7. Finally, let's test what happens when we have masks of different values on different interfaces. Configure the mask /24 in **virt1** and **virt3** and /25 in **virt2** and **virt4**. Discuss in detail what happens when you `ping` from **virt1** to the other machines and when you `ping` from **virt2** to the other machines.

Exercise3– (*) In this exercise, we keep using the "subnetting" scenario (Figure 2) that you have already running, to practice with indirect routes. The goal is to enable traffic between **virt2** and **virt4** while keeping their masks to /25. Notice that with this configuration, a `ping` from **virt2** to 192.168.0.224 (**virt4**) does not work directly. To enable IP traffic between **virt2** and **virt4**, we are going to try two different configurations with indirect routes (routers). In this

case, some **virtX** machines will be used as routers. To make a Linux act as a router, that is to say, to enable forwarding packets, we must execute the following command:

```
# echo 1 > /proc/sys/net/ipv4/conf/all/forwarding
```

1. In the first configuration, we will configure **virt1** and **virt3** with a mask /24. Notice that with this mask **virt1** and **virt3** can directly send packets to all the addresses in 192.168.0.0/24. Next, we will use these two machines as routers. To do so, enable forwarding in **virt1** and **virt3** and configure the appropriate routes in **virt2** and **virt4** (remember that these hosts have a mask /25).

   Note. Use network addresses in your `route` commands.

   Then, clear the ARP caches of **virt1**, **virt2** and **virt3**:

   ```
   # ip neigh flush all
   ```

   Finally, try a `ping` from **virt2** to **virt4**. Discuss the traffic that you observe on `SimNet0` and the entries of the ARP caches of **virt1**, **virt2**, **virt3** and **virt4**.

2. In the second configuration, we will use only **virt1** as router to enable IP traffic between **virt2** and **virt4**. Since **virt1** is going to be the router for both **virt2** and **virt4**, we need to configure **virt1** with two addresses, one on each /25 network. This can be done as follows:

   ```
   virt1$ ifconfig eth1 192.168.0.32/25
   virt1$ ifconfig eth1:0 192.168.0.232/25
   ```

   The second command creates the alias 0 for `eth1`, which is the way to configure a second IP address with `ifconfig`. The `eth1` interface of **virt1** has now two IPs: 192.168.0.32 and 192.168.0.232.

   Configure the proper network route on **virt2** and do the same in **virt4**.

   Clear the ARP caches of **virt1**, **virt2** and **virt3** with `ip neigh flush all`.

   Check your configuration trying a ping from **virt2** to **virt4**. Discuss the traffic that you observe on `SimNet0` and the entries of the ARP caches of **virt1**, **virt2** and **virt3**.

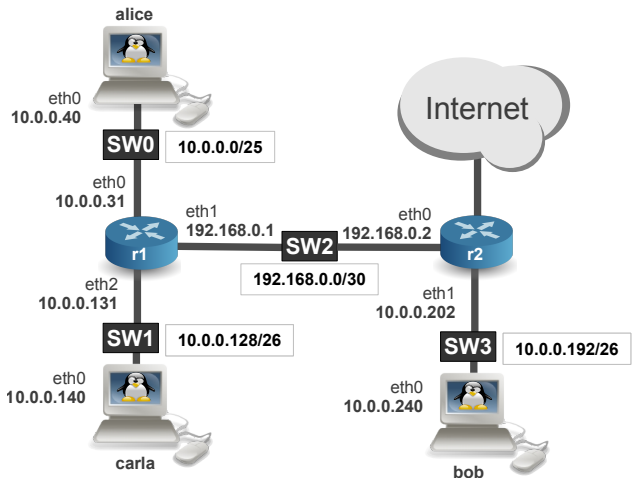


Figure 3: Scenario routing-abc.

Exercise4– Using the scenario routing-abc, follow the configuration explanations done in the class to configure step by step the topology of Figure 3. Finish the configuration to enable `pings` between **alice**, **bob** and **carla**.

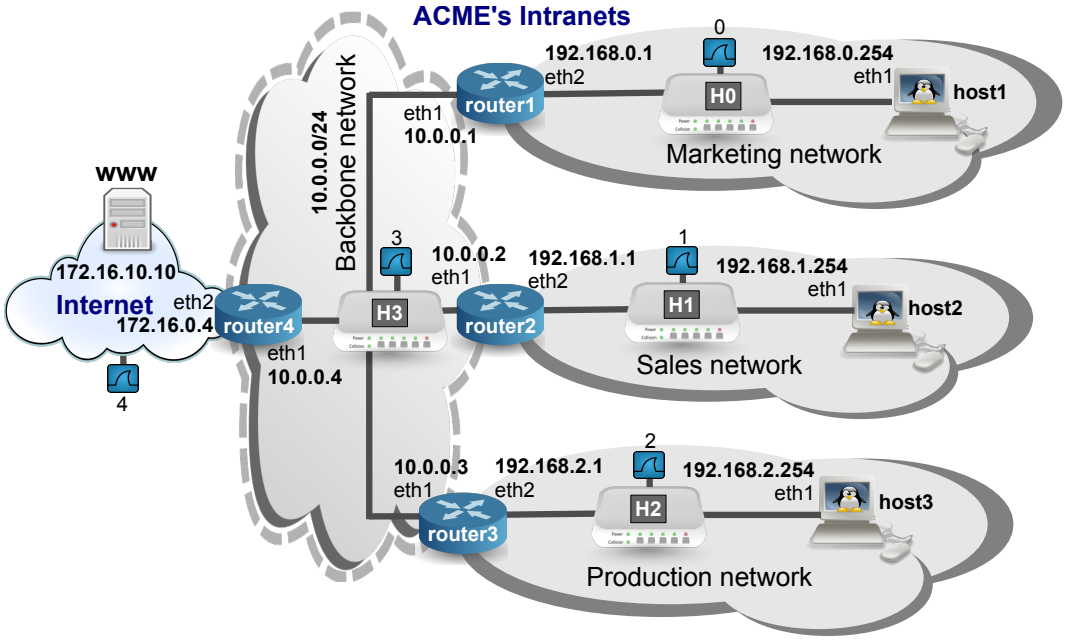


Figure 4: Scenario "routing".

Exercise5– In this exercise we will configure a network for a small fictitious company called ACME. Figure 4 shows the network topology. ACME has three departments: marketing, sales and production. Each department is represented by a host and a router. Finally, the IP network 10.0.0.0/24 interconnects the routers (backbone network). Type on your physical host the following command to start the scenario:

```
phyhost$ simctl routing start
```

1. Analyzing the IP addresses assigned in the network, select an appropriate netmask for each network interface of **host1**, **router1**, **host2** and **router2**. Verify the direct communications with `pings`. What would happen if you configure a mask /23 in `host1` (only in this host) and try a `ping` to **host2**?

   Note. You do not have to configure neither router3 nor host3 (they are already configured for the exercise).

Next, we will implement the minimum configuration required to enable communication between **host1** (marketing) and **host2** (sales). Capture in `SimNet0`, `SimNet1` and `SimNet2`, follow the next steps and answer each question.

2. Configure a route in **host1** to the 192.168.1.0/24 network. Send a `ping` of one message from **host1** to 192.168.1.254. Obviously, the `ping` does not succeed. Explain the traffic captured in each interface (frame fields, IP packet fields and ICMP message fields, MAC addresses, IP addresses, etc.).

3. Check that **router1** has enabled IP forwarding. Then, add the necessary entry in the routing table of **router1** to reach the sales network. Send again the `ping` from **host1** to 192.168.1.254. Obviously, the `ping` still does not succeed. Explain the traffic that you observe in each interface and the contents of the ARP cache of the different machines.

4. Finish the configuration adding entries for the network 192.168.0.0/24 in **router2** and **host2**. Check that the `ping` works correctly. Explain the IP addresses observed in each packet, the MAC addresses and the contents of the ARP cache of **host1**, **router1**, **host2** and **router2**.

5. Now, send a ping from **router1** to 192.168.1.254. Find out why it does not work and propose two configurations of the routing table of **host2** to fix the problem.

The current configuration of the backbone routers does not allow communication of hosts in sales or marketing departments with hosts on the production department (you can test that a `ping` from **host1** to **host3** does not succeed). This configuration is imposed by network administrators. However, we are going to show you that an advanced user placed on a host of the marketing or sales department can send and receive IP traffic to the production department under certain conditions. To illustrate this, we will use several `ping` commands that executed from **host1** can obtain responses from **host3**.

6. This exercise deals with the IPv4 **Source Routing** option. Capture on `SimNet0`, `SimNet3` and `SimNet2`, and execute the following commands:

```
host1$ ping -c1 -r  192.168.0.1  10.0.0.3  192.168.2.254
host1$ ping -c1 10.0.0.3  192.168.2.254
host1$ ping -c1 -r 10.0.0.3  192.168.2.254
```

   Explain in detail the output and what each command does. In addition, analyzing the IP datagrams captured, explain how the source routing pointer works and how the reverse path is stored.

In the last part of this practice we study a basic **Network Address Translation (NAT)**. NAT allows to exchange IP datagrams between private and public networks and vice versa. As you can observe in Figure 4, the IP networks of ACME use the private ranges of IP addressing 192.168.X.0/24 (with X=0,1,2) and 10.0.0.0/24. On the other hand, we will use **the range 172.16.0.0/16 to simulate public addressing**:

- We must stress that as you must know 172.16.0.0/16 is also private. However, for the purpose of this exercise, we will use addresses of 172.16.0.0/16 as if they were public.
- Following this criterion, the server **www** (a web server), which has address 172.16.10.10 is considered to be in the Internet with a public address. Following this idea, **www** is only able to forward IP datagrams to public addresses (addresses in the range 172.16.0.0/16).
- ACME, our company, is connected to the Internet using **router4**. This router has two network cards, `eth1` configured with a private address (10.0.0.4) and `eth2` configured with a simulated public address (172.16.0.4).
- In the simple NAT that we study in this exercise, called **static NAT**, **router4** will enable the communication of **host1** with **www** by performing a one-to-one IP translation. In particular, **router4** will NAT the private IP address of **host1**, 192.168.0.254, to the simulated public address 172.16.0.254.

To start this exercise, set the proper configuration typing the following command on your physical host:

```
phyhost$ simctl routing exec netconf
```

7. Capture on `SimNet4` and try a `ping` from **host1** to 172.16.10.10 (**www**). Explain the traffic captured and why do you think that the `ping` does not work?

8. Execute the following commands in **router4**:

```
r4# iptables -t nat -F
r4# iptables -t nat -A POSTROUTING -s 192.168.0.254 -o eth2 -j SNAT --to-source 172.16.0.254
```

   - The first command clears the NAT tables.
   - The second adds a rule so that datagrams of communications originated with IP address 192.168.0.254 get their source address modified and translated to 172.16.0.254 (and does the reverse operation when the datagram goes in the other way).

   Capture on `SimNet4` and try a `ping` from **host1** to **www**. Why do you think the `ping` still does not work? move to the next exercise to find the answer.

9. Add the public address 172.16.0.254 in **router4** as the alias eth2:0:

```
r4# ifconfig eth2:0 172.16.0.254/16
```

Test your configuration with a `ping` and also with a `lynx` from **host1**:

```
host1# ping -c1 172.16.10.10
host1# lynx http://172.16.10.10
```

Note that the NAT rule also changes the destination address in the packets going from **www** to **host1**.

Finally, explain what type of application is `lynx`.

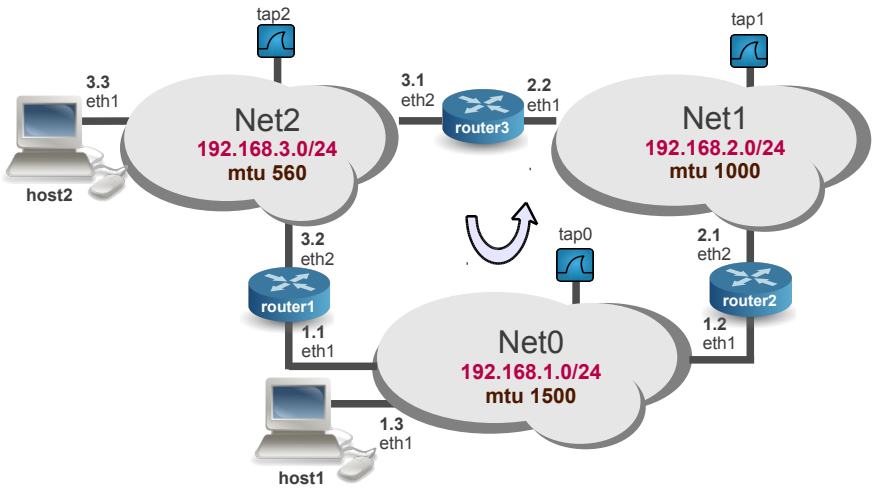


Figure 5: Network Topology for ICMP Testing.

Exercise6– The goal of this exercise is to practice with the fragmentation of IP datagrams and with the operation of various ICMP messages when there are different error conditions. The network used for this exercise is shown in Figure 5. Type on your physical host the following command to start the scenario:

```
phyhost$ simctl icmp start
```

Figure 5 shows three networks (*Net0*, *Net1* and *Net2*), interconnected with three routers (**router1**, **router2** and **router3**). Notice that the MTU of each network is different. The MTU of an Ethernet interface can be decreased (under 1500) with the `ifconfig` command (see the `man` of `ifconfig`).

Configure the routers with the parameters shown in the following tables:

router1	IP	MTU
eth1	192.168.1.1/24	1500
eth2	192.168.3.2/24	560

router2	IP	MTU
eth1	192.168.1.2/24	1500
eth2	192.168.2.1/24	1000

router3	IP	MTU
eth1	192.168.2.2/24	1000
eth2	192.168.3.1/24	560

Then, configure **host1** and **host2** as follows:

host1	IP	MTU
eth1	192.168.1.3/24	1500

host2	IP	MTU
eth1	192.168.3.3/24	560

Finally, you have to configure the appropriate routes for indirect delivery as follows:

- **router1** has its default route through **router2**.
- **host1** has its default route through **router2**.

- **router2** has its default route through **router3**.
- **router3** has its default route through **router1**.
- **host2** has its default route through **router1**.

A. Fragmentation at Origin

1. Find out which is the path that a packet will take going from **host1** to **host2**, indicating the networks and routers that it will cross.
2. Find out the path that a packet will take going from **host2** to **host1**, indicating the networks and routers that it will cross.
3. Check your previous answers capturing traffic on `tap0`, `tap1` and `tap2` and executing the following `pings`:

```
host1# ping -c 1 192.168.3.3
host2# ping -c 1 192.168.1.3
```

4. Determine the size of the IP packets containing the ICMP *echo-request* and *echo-reply* messages. Was it necessary to fragment any IP packet somewhere in the network?
5. Comment the value of the DF flag found in the IP headers of captured packets. Which is the purpose of this flag?

Now, capturing traffic on the three *taps* send two *echo-request* messages of 900 bytes of payload from **host1** to **host2**. **Note. Always delete the routing cache before sending the `ping`:**

```
host1# ip route flush cache
host1# ping -c 2 -s 900 192.168.3.3
```

Analyzing the traffic captured answer the following questions:

6. Which is the size of the first IP packet captured on `tap0`? Find the sizes of the headers of each protocol found in the frame that encapsulates this packet. Identify where are placed the 900 bytes indicated in the `ping` command.
7. Checking the captures on `tap1` and `tap2` try to find out which is the path that the previous packet has followed.
8. Analyze the ICMP packet "*Destination unreachable*". This ICMP message is telling us that the destination is unreachable, but why? Analyze the ICMP header of this message. Which is the IP packet that caused the error? Who is the sender of this ICMP message? Who is the recipient? Which path has followed this ICMP message from source to destination?

You should have observed that the first `echo-request` message with 900 bytes of payload has not reached the destination and, therefore, there was not an `echo-reply`. Now, you have to analyze the captures for the second *echo-request*.

9. Comment the values of the "Don't Fragment" (DF), " More Fragments" (MF) flags, "identification" (ID), "fragment offset" (FO) and the size of each IP packet related to this second ICMP message. Which is the purpose of MF, ID and FO? Try to correlate what you observe with the fact that we send an *echo-request* with 900 bytes of payload and that there is an IP network with an MTU of 560 bytes.
10. Identify the path followed by the fragmented *echo-request* ICMP message from origin to destination and identify as well the path followed by *echo-reply* ICMP response message from origin to destination. Which machine made packet fragmentation?

B. Fragmentation by Routers

11. Capture traffic on the three `tap` interfaces and send just one *echo-request* message from **host1** to **host2** with a payload of 900 bytes but with DF=0 (see with `man` the -M option of `ping`). Analyzing the captured traffic, determine where fragmentation is occurring.

12. What happens if we send one *echo-request* message from **host1** to **host2** with a payload of 1200 bytes with DF=0?

C. *Time To Live (TTL) Exceeded*

The goal of this test is to generate the error condition that causes the transmission of a Time To Live exceeded ICMP message. Recall that when an IP datagram arrives at a router, before being forwarded to destination, the router must do some processing:

- Decrement the Time To Live (TTL) field by one.
- Recalculate the "checksum" field (given that the TTL has changed).
- If the TTL reaches zero, the router throws away the packet and sends a Time To Live exceeded ICMP message to the sender of the IP datagram that generated the error.

To conveniently check the operation described above, we will send an *echo-request* with TTL=8 (see -t option in the man page of `ping`) from **host1** to the IP address 10.0.0.1. Before starting the practical test, answer theoretically the following questions:

13. Given the configuration of the routers and hosts, which is the path that a datagram will follow in our network from **host1** to 10.0.0.1? If TTL=8, which router will detect the error condition?

14. If the router that produces the error condition sends the *Time To Live exceeded* ICMP message to `host1`, which path will this packet follow? Which will be the source IP address of this datagram?

Now, execute the `ping` command from **host1**.

15. Capture on the three `tap` interfaces and explain the captured traffic.

16. What happens if we set TTL=9?